Tutorial

How To Install Python 3 and Set Up a Programming Environment on Ubuntu 22.04

Published on April 26, 2022

Development Ubuntu Ubuntu 22.04 Python

By Alex Garnett
Senior DevOps Technical Writer

Table of contents

Popular topics

Introduction

The Python programming language is an increasingly popular choice for both beginners and experienced developers. Flexible and versatile, Python has strengths in scripting, automation, data analysis, machine learning, and back-end development. First published in 1991 with a name inspired by the British comedy group Monty Python, the development team wanted to make Python a language that was fun to use.

This tutorial will get your Ubuntu 22.04 server set up with a Python 3 programming environment. Programming on a server has many advantages and supports collaboration across development projects. The general principles of this tutorial will apply to any distribution of Debian Linux.

Prerequisites

In order to complete this tutorial, you should have a non-root user with `sudo` privileges on an Ubuntu 22.04 server. To learn how to achieve this setup, follow our initial server setup guide.

If you're not already familiar with a terminal environment, you may find the article An Introduction to the Linux Terminal useful for becoming better oriented with the terminal.

Step 1 – Setting Up Python 3

Ubuntu 22.04 and other versions of Debian Linux ship with Python 3 pre-installed. To make sure that our versions are up-to-date, update your local package index:

```
$ sudo apt update
```

Then upgrade the packages installed on your system to ensure you have the latest versions:

```
$ sudo apt -y upgrade
```

The `-y` flag will confirm that we are agreeing for all items to be installed, but depending on your version of Linux, you may need to confirm additional prompts as your system updates and upgrades.

Once the process is complete, we can check the version of Python 3 that is installed in the system by typing:

```
$ python3 -V
```

You'll receive output in the terminal window that will let you know the version number. While this number may vary, the output will be similar to this:

```
Output
Python 3.10.2+
```

To manage software packages for Python, let's install **pip**, a tool that will install and manage programming packages we may want to use in our development projects. You can learn more about modules or packages that you can install with pip by reading How To Import Modules in Python 3.

```
$ sudo apt install -y python3-pip
```

Python packages can be installed by typing:

```
$ pip3 install package_name
```

Here, `package_name` can refer to any Python package or library, such as Django for web development or NumPy for scientific computing. So if you would like to install NumPy, you can do so with the command `pip3 install numpy`.

There are a few more packages and development tools to install to ensure that we have a robust setup for our programming environment:

```
$ sudo apt install -y build-essential libssl-dev libffi-dev python3-dev
```

Once Python is set up, and pip and other tools are installed, we can set up a virtual environment for our development projects.

Step 2 – Setting Up a Virtual Environment

Virtual environments enable you to have an isolated space on your server for Python projects, ensuring that each of your projects can have its own set of dependencies that won't disrupt any of your other projects.

Setting up a programming environment provides greater control over Python projects and over how different versions of packages are handled. This is especially important when working with third-party packages.

You can set up as many Python programming environments as you would like. Each environment is basically a directory or folder on your server that has a few scripts in it to make it act as an environment.

While there are a few ways to achieve a programming environment in Python, we'll be using the **venv** module here, which is part of the standard Python 3 library. Let's install venv by typing:

```
$ sudo apt install -y python3-venv
```

With this installed, we are ready to create environments. Let's either choose which directory we would like to put our Python programming environments in, or create a new directory with `mkdir`, as in:

```
$ mkdir environments
```

Then navigate to the directory where you'll store your programming environments:

```
$ cd environments
```

Once you are in the directory where you would like the environments to live, you can create an environment by running the following command:

```
$ python3 -m venv my_env
```

Essentially, `pyvenv` sets up a new directory that contains a few items which we can view with the `ls` command:

```
$ ls my_env
```

```
Output
bin include lib lib64 pyvenv.cfg share
```

Together, these files work to make sure that your projects are isolated from the broader context of your server, so that system files and project files don't mix. This is good practice for version control and to ensure that each of your projects has access to the particular packages that it needs. Python Wheels, a built-package format for Python that can speed up your software production by reducing the number of times you need to compile, will be in the Ubuntu 22.04 `share` directory.

To use this environment, you need to activate it, which you can achieve by typing the following command that calls the **activate** script:

```
$ source my_env/bin/activate
```

Your command prompt will now be prefixed with the name of your environment, in this case it is called `my_env`. Depending on what version of Debian Linux you are running, your prefix may appear somewhat differently, but the name of your environment in parentheses should be the first thing you see on your line:

```
(my_env) sammy@ubuntu:~/environments$
```

This prefix lets us know that the environment `my_env` is currently active, meaning that when we create programs here they will use only this particular environment's settings and packages.

Note: Within the virtual environment, you can use the command `python` instead of `python3`, and `pip` instead of `pip3` if you would prefer. If you use Python 3 on your machine outside of an environment, you will need to use the `python3` and `pip3` commands exclusively.

After following these steps, your virtual environment is ready to use.

Step 3 – Creating a "Hello, World" Program

Now that we have our virtual environment set up, let's create a traditional "Hello, World!" program. This will let us test our environment and provides us with the opportunity to become more familiar with Python if we aren't already.

To do this, we'll open up a command line text editor such as `nano` and create a new file:

```
(my_env) sammy@ubuntu:~/environments$ nano hello.py
```

Once the text file opens up in the terminal window we'll type out our program:

hello.py

```
print("Hello, World!")
```

Save the file and exit `nano` by pressing `CTRL + X`, `Y`, and then `ENTER`.

Once you exit out of the editor and return to your shell, you can run the program:

```
(my_env) sammy@ubuntu:~/environments$ python hello.py
```

The `hello.py` program that you created should cause your terminal to produce the following output:

```
Output
Hello, World!
```

To leave the environment, type the command `deactivate` and you will return to your original directory.

Conclusion

Congratulations! At this point you have a Python 3 programming environment set up on your Ubuntu Linux server and you can now begin a coding project!

If you are using a local machine rather than a server, refer to the tutorial that is relevant to your operating system in our How To Install and Set Up a Local Programming Environment for Python 3 series.

With your server ready for software development, you can continue to learn more about coding in Python by reading our free How To Code in Python 3 eBook, or consulting our Python tutorials.

About the author

Alex Garnett Author
Senior DevOps Technical Writer

See author profile

Former Senior DevOps Technical Writer at DigitalOcean. Expertise in topics including Ubuntu 22.04, Linux, Rocky Linux, Debian 11, and more.

Category: Tutorial

Tags: Development Ubuntu Ubuntu 22.04 Python

Sign in/up to comment

kurt krueckeberg
July 4, 2022

Show less

Shouldn’t

```
$ source my_env/bin/activate
```

be

```
$ source environments/my_env/bin/activate
```

Reply

The Wave has everything you need to know about building a business, from raising funding to marketing your product.

Learn more →

Get our newsletter

Stay up to date by signing up for DigitalOcean's Infrastructure as a Newsletter.

Email address

Submit

New accounts only. By submitting your email you agree to our Privacy Policy

The developer cloud

Scale up as you grow — whether you're running one virtual machine or ten thousand.

View all products

Get started for free

Sign up and get $200 in credit for your first 60 days with DigitalOcean.*

Get started

*This promotional offer applies to new accounts only.

Identity and Access Management (IAM)

Cloudways

View all Products

 Sitemap.